

Java 객체지향 프로그래밍 핵심

클래스, 객체, 상속, 캡슐화와 오버로딩/오버라이딩을 학습합니다.

1. 객체지향 프로그래밍의 기본 개념

객체지향 프로그래밍(OOP)은 프로그램을 서로 협력하는 객체들의 집합으로 바라보는 방식이다. 객체는 상태를 나타내는 필드와 동작을 나타내는 메서드를 가진다.

클래스와 객체

- 클래스(Class): 객체를 만들기 위한 설계도이다. 어떤 데이터와 기능을 가질지 정의한다.
- 객체(Object): 클래스를 기반으로 실제 메모리에 생성된 인스턴스이다.

캡슐화와 상속

- 캡슐화: 객체 내부 상태를 직접 노출하지 않고 메서드를 통해 접근하도록 하여 변경 영향을 줄인다.
- 상속: 기존 클래스의 필드와 메서드를 하위 클래스가 재사용하거나 확장하는 기능이다.

상속은 코드 재사용에 도움이 되지만, 지나친 상속은 클래스 간 결합도를 높일 수 있으므로 역할과 책임을 고려해 사용해야 한다.

핵심 정리: 클래스는 설계도, 객체는 실제 인스턴스이며, 캡슐화는 내부 상태를 보호하고 상속은 기존 기능을 재사용하거나 확장한다.

학습 질문 예시: "클래스와 객체의 차이는 뭐야?", "캡슐화가 왜 필요해?"

Java 객체지향 프로그래밍 핵심

클래스, 객체, 상속, 캡슐화와 오버로딩/오버라이딩을 학습합니다.

2. 오버로딩과 오버라이딩

오버로딩(Overloading)은 같은 클래스 안에서 같은 이름의 메서드를 매개변수의 개수나 타입을 다르게 하여 여러 개 정의하는 것이다. 어떤 메서드를 호출할지는 컴파일 시점에 결정된다.

오버라이딩(Overriding)은 상위 클래스가 가진 메서드를 하위 클래스에서 같은 시그니처로 다시 정의하는 것이다. 실행 시 실제 객체의 타입에 따라 호출될 메서드가 결정될 수 있다.

구분	오버로딩	오버라이딩
목적	같은 기능 이름을 다양한 입력으로 제공	상위 클래스의 동작을 하위 클래스에 맞게 재정의
위치	주로 같은 클래스 내부	상속 관계의 상위/하위 클래스
조건	매개변수 개수 또는 타입이 달라야 함	메서드 이름과 매개변수 구성이 동일해야 함
결정 시점	컴파일 시점	실행 시점의 다형성과 연관

예시

`print(int value)`와 `print(String value)`를 함께 정의하면 오버로딩이다. 반면 `Animal`의 `sound()`를 `Dog`가 `sound()`로 다시 구현하면 오버라이딩이다.

핵심 정리: 오버로딩은 "같은 이름, 다른 매개변수", 오버라이딩은 "상위 메서드를 하위 클래스에서 재정의"이다.

학습 질문 예시: "오버로딩과 오버라이딩 차이가 뭐야?"